RAPPORT D'AUDIT

CYBERSÉCURITÉ — SCRIPT PYTHON AEGIS

Référence	TS-2026-SSI-001
Client	PME TechSud
Auditeur	Groupe AEGIS — IPSSI BTC1
Date	16 avril 2026
Version	**1.0.0 — CONFIDENTIEL**

Sommaire

1.Résumé exécutif
2.Contexte et périmètre de l'audit
3.Méthodologie
4.Analyse détaillée des modules
5.Synthese des constats et cartographie des risques
6.Plan de recommandations
7.conclusion
8.Annexes

1. Résumé exécutif

Le script Python AEGIS (v1.0.0) est un outil d'audit automatisé de la sécurité d'une infrastructure Linux, développé dans le cadre du projet BTC1 à l'IPSSI. Il cible l'infrastructure simulée de la PME TechSud, composée de six hôtes (serveurs de production, web, base de données, pare-feu, NAS, poste administrateur).

L'analyse couvre 10 domaines : inventaire système, scan de ports, configuration SSH, pare-feu UFW, fail2ban, gestion des comptes, détection d'IoC, mises à jour, services réseau actifs et score de conformité SSI global.

Points forts identifiés :

- Architecture modulaire bien structurée (10 fonctions distinctes, responsabilité unique).
- Gestion robuste des erreurs : try/except systématiques dans run_cmd et les parseurs de fichiers.
- Score de conformité calculé automatiquement avec pondération par domaine.
- Export dual JSON + CSV pour intégration dans un rapport ou un SIEM.
- Détection d'IoC contextualisée (fichiers suspects, crons malveillants, ports C2).

Risques majeurs identifiés :

- Pas d'analyse des logs système (journalctl / auth.log) — vecteur d'attaque SSH non couvert.
- Aucune vérification des permissions fichiers (SUID/SGID, world-writable).
- Détection CIDR non implémentée : cibles réseau limitées à une IP unique.
- Absence de tests unitaires malgré la structure modulaire.

Niveau de risque global
⚠ MOYEN — Améliorations requises

2. Contexte et périmètre de l'audit

2.1 Contexte de la mission

Suite à un incident de sécurité détecté sur l'infrastructure TechSud, le groupe AEGIS a développé un script d'audit automatisé (audit_aegis.py) permettant de vérifier rapidement l'état de sécurité des serveurs Linux Debian 12 constituant cette infrastructure. Le présent rapport analyse la qualité, la complétude et la conformité de ce script vis-à-vis des bonnes pratiques SSI.

2.2 Périmètre technique

2.3 Référentiels utilisés

- ISO 27001 — Système de management de la sécurité de l'information
- Guide d'hygiène informatique ANSSI — 42 mesures de sécurité
- CIS Benchmarks Debian Linux — Durcissement de configuration
- OWASP Top 10 — Pour les vérifications de services web

3. Méthodologie

L'audit repose sur une revue statique du code source Python combinée à une analyse dynamique simulée. Chaque module a été évalué selon quatre critères : complétude fonctionnelle, qualité du code, gestion des erreurs et adéquation avec les bonnes pratiques de sécurité.

Phase	Description
Revue de code	Lecture intégrale du script (748 lignes) — analyse structure, logique, sécurité
Analyse des modules	Évaluation de chacun des 10 modules d'audit selon les critères définis
Détection des lacunes	Comparaison avec la méthodologie de référence (fichier Methodologie_Rapport_audit)
Scoring	Attribution d'un score de conformité et d'un niveau de criticité par constat

4. Analyse détaillée des modules

4.1 Module SSH — audit_ssh()

Le module vérifie 7 paramètres de sshd_config : PermitRootLogin, PasswordAuthentication, PermitEmptyPasswords, X11Forwarding, Protocol, Port et MaxAuthTries. La logique de vérification est claire et bien commentée.

Paramètre vérifié	Statut	Observation
PermitRootLogin	OK	Contrôle des valeurs yes / without-password / prohibit-password
PasswordAuthentication	OK	Vérification correcte
PermitEmptyPasswords	OK	Présent — absent du document de référence, valeur ajoutée
X11Forwarding	OK	Correctement vérifié
Protocol	OK	Détecte SSHv1 — bonne pratique
Port SSH (!=22)	OK	Recommandation non-standard incluse
MaxAuthTries	OK	Seuil fixé à <= 4, conforme aux recommandations
AllowAgentForwarding	MOYENNE	Non vérifié — présent dans la référence pédagogique

```python
def audit_ssh() -> list[dict]:
    """Analyse la configuration SSH du serveur local."""
    titre_section("3. Vérification SSH")
    chemin_config = "/etc/ssh/sshd_config"
    verifications = []
    if not os.path.exists(chemin_config):
        warn(f"{chemin_config} introuvable — SSH non installé ou chemin différent")
        return verifications
    try:
        with open(chemin_config, "r") as f:
            contenu = f.read()
    except PermissionError:
        warn("Impossible de lire sshd_config — relancez avec sudo")
        return verifications
```

4.2 Module Pare-feu — audit_firewall()

Le module détecte si ufw est actif, extrait les règles et alerte sur les règles permissives (ALLOW Anywhere). La détection de la politique par défaut (deny incoming) mentionnée dans le document de référence est partiellement absente.

Vérification	Statut	Observation
UFW actif	OK	Détection correcte via ufw status verbose
Règles permissives ALLOW Anywhere	OK	Alerte générée sur les règles trop ouvertes
Politique par défaut deny incoming	MOYENNE	Contrôle absent — non testé explicitement

Ports exposés Internet	MOYENNE	Pas de croisement avec la liste PORTS_DANGEREUX via ufw

```python
def audit_firewall() -> dict:
    """Vérifie l'état et les règles du pare-feu ufw."""
    titre_section("4. Vérification du pare-feu (ufw)")
    resultat = {
        "outil":   "ufw",
        "actif":   False,
        "regles":  [],
        "alertes": [],
    }
    code, stdout, stderr = run_cmd(["ufw", "status", "verbose"])
    if code == -1 and "introuvable" in stderr:
        warn("ufw non installé — sudo apt install ufw")
        resultat["alertes"].append("ufw non installé")
        return resultat
    if "inactive" in stdout.lower():
        err("Le pare-feu ufw est INACTIF")
        resultat["alertes"].append("pare-feu inactif")
        return resultat
    resultat["actif"] = True
    ok("ufw est actif")
    # Extraction des règles
    for ligne in stdout.splitlines():
        ligne = ligne.strip()
        if ligne and not ligne.startswith("Status") and not ligne.startswith("To"):
            resultat["regles"].append(ligne)
            if "ALLOW" in ligne.upper() and "Anywhere" in ligne:
                warn(f"Règle permissive détectée : {ligne}")
                resultat["alertes"].append(f"Règle permissive : {ligne}")
            else:
                info(f"Règle : {ligne}")
    return resultat
```

4.3 Module fail2ban — audit_fail2ban()

Le module vérifie l'installation, l'état du service et la liste des jails actives. L'alerte sur l'absence de la jail sshd est pertinente. Un enrichissement possible serait la lecture des statistiques de bannissement (tentatives bloquées par IP).

Vérification	Statut	Observation
fail2ban installé et actif	OK	Détection via fail2ban-client status
Jail sshd présente	OK	Alerte si absente — bonne pratique
Statistiques de bannissements	BASSE	Non récupérées — amélioration possible
Lecture fail2ban.log	MOYENNE	Absent — référencé dans la liste des logs clés

```python
def audit_fail2ban() -> dict:
    """Vérifie l'état de fail2ban et les jails actives."""
    titre_section("5. Vérification de fail2ban")
    resultat = {
        "installe":    False,
        "actif":       False,
        "jails":       [],
        "alertes":     [],
    }
    code, stdout, _ = run_cmd(["fail2ban-client", "status"])
    if code == -1:
        err("fail2ban non installé ou non démarré")
        resultat["alertes"].append("fail2ban absent ou inactif")
        return resultat
```

4.4 Module Utilisateurs — audit_utilisateurs()

Module solide : il parcourt /etc/passwd via pwd.getpwall(), détecte les membres sudo/wheel, vérifie l'existence des répertoires home et signale les comptes sensibles (www-data, nobody, deploy) avec un shell actif.

Vérification	Statut	Observation
Comptes avec shell actif	OK	Filtrage correct UID >= 1000
Membres groupe sudo/wheel	OK	Alerte générée — bonne pratique
Comptes sensibles (www-data…)	OK	Contrôle spécifique sur 3 comptes à risque
Comptes UID 0 multiples	CRITIQUE	Non vérifié — vecteur d'attaque critique (backdoor root)
Comptes sans mot de passe	HAUTE	Non vérifié — nécessite lecture /etc/shadow
Comptes inactifs	MOYENNE	Non détectés (last/lastlog non utilisés)

```
    code, out, _ = run_cmd(["groups", nom])
    in_sudo = "sudo" in out or "wheel" in out
    if in_sudo:
        alertes.append("membre du groupe sudo")
u = {
    "nom":          nom,
    "uid":          uid,
    "shell":        shell,
    "home":         home,
    "peut_login":   peut_login,
    "est_systeme":  est_systeme,
    "alertes":      alertes,
}
utilisateurs.append(u)
if peut_login and uid >= 1000:
    if alertes:
        warn(f"Compte : {nom} (UID {uid}) — {', '.join(alertes)}")
    else:
        ok(f"Compte : {nom} (UID {uid}) — shell : {shell}")
```

4.5 Module Fichiers suspects — audit_fichiers_suspects()

Ce module est contextualisé à l'incident TechSud : il vérifie la présence de 3 IoC connus et analyse les tâches cron à la recherche de patterns suspects (wget, curl, base64, nc, reverse shell bash).

Vérification	Statut	Observation
IoC fichiers connus (/tmp, /var/www, /etc/cron.d)	OK	3 fichiers ciblés — pertinents au scénario TechSud
Patterns cron malveillants	OK	9 patterns regex couvrent les TTP courants (T1053 MITRE ATT&CK)
Fichiers SUID/SGID	CRITIQUE	Absent — vecteur d'escalade de privilèges majeur
Fichiers world-writable	CRITIQUE	Absent — exposé dans la méthodologie de référence
Permissions /etc/shadow, /etc/passwd	HAUTE	Non vérifiées

```
for chemin in FICHIERS_SUSPECTS:
    existe = os.path.exists(chemin)
    entree = {
        "type":       "fichier_ioc",
        "chemin":     chemin,
        "present":    existe,
        "criticite":  "CRITIQUE" if existe else "OK",
    }
    trouvailles.append(entree)
    if existe:
        err(f"IoC détecté : {chemin}")
    else:
        ok(f"Absent (OK) : {chemin}")
```

4.6 Module Logs — ABSENT

⚠ CONSTAT CRITIQUE : Aucun module d'analyse de logs n'est implémenté dans le script AEGIS.

Le document de référence identifie l'analyse des logs comme un module essentiel (audit_logs.py) couvrant : tentatives SSH échouées (journalctl, grep auth.log), connexions root réussies (last root), commandes sudo suspectes, erreurs critiques système (journalctl -p crit).

L'absence de ce module constitue une lacune fonctionnelle majeure, car les logs sont la principale source de détection d'intrusion post-incident.

4.7 Module Services — audit_services_actifs()

Le module utilise ss (avec fallback netstat) pour lister les ports en écoute et croise avec PORTS_DANGEREUX. Une redondance mineure est présente ligne 554 (appel run_cmd inutile pour vérifier la présence de ss).

Vérification	Statut	Observation
Ports TCP en écoute	OK	ss avec fallback netstat — robuste
Croisement PORTS_DANGEREUX	OK	Alerte sur ports connus (4444, 3306, RDP…)
Ligne 554 — redondance run_cmd	BASSE	Appel inutile à which ss dans la boucle — impact mineur

```python
def audit_services_actifs() -> list[dict]:
    """Liste les services réseau en écoute sur la machine locale."""
    titre_section("9. Services réseau en écoute")
    services = []
    code, stdout, _ = run_cmd(["ss", "-tlnp"])
    if code != 0:
        code, stdout, _ = run_cmd(["netstat", "-tlnp"])
    for ligne in stdout.splitlines():
        if "LISTEN" not in ligne and "ESTAB" not in ligne:
            continue
        parts = ligne.split()
        if len(parts) < 4:
```

5. Synthèse des constats et cartographie des risques

ID	Constat	Criticité	Module	Effort remédiation
C-01	Module logs absent (journalctl, auth.log)	CRITIQUE	Absent	Moyen (2–4h)
C-02	Aucun contrôle fichiers SUID/SGID/world-writable	CRITIQUE	audit_fichiers_suspects	Faible (1–2h)
C-03	Comptes UID 0 multiples non détectés	CRITIQUE	audit_utilisateurs	Faible (< 1h)
H-01	Comptes sans mot de passe non détectés	HAUTE	audit_utilisateurs	Moyen (1–2h)
H-02	Permissions /etc/shadow non vérifiées	HAUTE	audit_fichiers_suspects	Faible (< 1h)
M-01	Politique pare-feu default deny non testée	MOYENNE	audit_firewall	Faible (< 1h)
M-02	Plage CIDR non supportée (cible unique)	MOYENNE	audit_ports / main	Moyen (2–3h)
M-03	AllowAgentForwarding SSH non vérifié	MOYENNE	audit_ssh	Faible (< 30min)
M-04	Comptes inactifs non détectés (lastlog)	MOYENNE	audit_utilisateurs	Faible (1h)
B-01	Absence de tests unitaires	BASSE	Projet global	Élevé (4–8h)
B-02	Ligne 554 : redondance run_cmd dans boucle	BASSE	audit_services_actifs	Faible (< 15min)
B-03	Aucun rapport HTML généré	BASSE	report_generator	Moyen (2–4h)

Répartition par criticité :

CRITIQUE : 3	HAUTE : 2	MOYENNE : 4	BASSE : 3

6. Plan d'action et recommandations

6.1 Actions prioritaires

Réf	Action	Module	Impact attendu
C-01	Implémenter audit_logs() : journalctl SSH, auth.log, last root, sudo suspicious	Nouveau module	Détection intrusion SSH
C-02	Ajouter find / -perm -4000 (SUID) et find / -perm -o+w (world-writable)	audit_fichiers_suspects	Détection escalade privil.
C-03	Filtrer /etc/passwd sur UID == 0 et alerter si > 1 compte	audit_utilisateurs	Détection backdoor root
H-01	Lire /etc/shadow, détecter champs password vides ('!!' ou '*' = OK)	audit_utilisateurs	Comptes compromissibles

6.2 Améliorations

Réf	Action	Module	Bénéfice
M-01	Tester explicitement Default: deny (incoming) dans ufw status	audit_firewall	Conformité politique pare-feu
M-02	Implémenter la résolution CIDR avec ipaddress.ip_network()	audit_ports / main	Scan de plages réseau réel
M-03	Ajouter AllowAgentForwarding no à la liste EXPECTED dans audit_ssh	audit_ssh	Couverture SSH complète
B-01	Écrire tests unitaires pytest avec mocking subprocess (unittest.mock)	tests/test_modules.py	Fiabilité & maintenabilité
B-03	Ajouter un générateur de rapport HTML avec Jinja2 ou f-strings	report_generator	Rapport lisible en navigateur

7. Conclusion

Le script AEGIS représente un travail sérieux et bien structuré. La qualité du code est au-dessus de la moyenne pour un projet académique : typage, docstrings, gestion des erreurs, architecture modulaire et export multi-format sont des points forts notables.

Le groupe AEGIS dispose d'une base solide pour produire un outil d'audit de production. Les recommandations prioritaires du plan d'action (section 6.1) devraient être traitées avant la soutenance finale.

8. Annexes

8.1 Modules du script AEGIS — Récapitulatif

Module	Fonction	Lignes	Points forts
Système	audit_systeme()	158–186	Collecte complète (hostname, IP, OS, kernel, uptime)
Ports	audit_ports()	199–229	13 ports, croisement PORTS_DANGEREUX
SSH	audit_ssh()	234–311	7 vérifications, logique check() réutilisable
Firewall	audit_firewall()	316–346	UFW, règles permissives
fail2ban	audit_fail2ban()	351–383	Jails, alerte sshd manquante
Utilisateurs	audit_utilisateurs()	388–438	sudo, shell, home, comptes sensibles
Fichiers IoC	audit_fichiers_suspects()	443–489	Fichiers ciblés + patterns cron
MAJ	audit_mises_a_jour()	494–535	apt + dnf, simulation upgrade
Services	audit_services_actifs()	541–571	ss/netstat, croisement dangereux
Score	calcul_score()	576–625	Score pondéré 100pts, niveau BON/MOYEN/INSUFFISANT
LOGS	— ABSENT —	—	À implémenter en priorité

8.2 Références

- ANSSI — Guide d'hygiène informatique (https://cyber.gouv.fr)
- CIS Debian Linux Benchmark — https://www.cisecurity.org
- MITRE ATT&CK — Techniques T1053 (cron), T1078 (Valid Accounts), T1021 (SSH)
- Python Subprocess Security — https://docs.python.org/3/library/subprocess.html
- Document pédagogique : Methodologie_Rapport_audit.pdf — IPSSI 2026